Lesson 1 — Introduction and the ML Workflow

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

25 September 2026

Welcome

- 10 lessons, 3 hours each, every Friday until 27 November
- A first course in **machine learning**, for people who can already program
- Foundations, not products

What we will cover

1–2	Workflow, data preparation
3–4	Regression, classification
5	**Experimental methodology**
6–7	k-NN, SVM, trees, ensembles
8	Unsupervised learning
9–10	Neural networks, CNNs

What this course is not

- Not a tour of current AI products
- Not a library tutorial
- Not a course where you call `.fit()` and report the number

Your advantage, and your gap

Advantage: you can program. You will implement methods, not only call them.

Gap: you can produce results faster than you can judge them.

How you will be assessed

- **Weekly exercises** — set every Friday, due the next
- **Final project** — an end-to-end study, with peer review
- **Written exam** — closed book, drawn from the handouts

The three things you get each lesson

- **Handout** — the reference text, with the mathematics
- **Slides** — what we do in the room
- **Notebooks** — runnable implementations

Plus a quiz, and the exercise.

Getting the material

```
git clone https://github.com/fabioantonini/\
technologies-for-artificial-intelligence.git
cd technologies-for-artificial-intelligence
docker compose up
```

Then `http://127.0.0.1:8888/lab?token=aicourse`

Today

- What learning from data means
- A short history, and what it teaches
- The three kinds of learning
- The end-to-end workflow
- Four ways a model misleads you

A problem you cannot specify

Write a program that decides whether an email is spam.

- Filter certain words? Spam adapts.
- Filter certain senders? They change.
- The exceptions grow until nobody can maintain it

The inversion

You cannot state the rule. But you can **recognise** the answer.

So: supply examples, and let a procedure find a rule consistent with them.

Making it precise

- Inputs from a space X, outputs from a space Y
- Pairs (x, y) drawn from an unknown distribution D
- A loss L(ŷ, y): the cost of answering ŷ when the truth is y
- Goal: find f: $X \to Y$ that predicts well

What we actually want

The average loss over all data the world might produce — including data that does not exist yet:

$$R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\left[L(f(x), y)\right]$$

What we can actually compute

Average loss over the finite sample we happen to hold:

$$\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^{n} L(f(x_i), y_i)$$

The gap that explains everything

- We **minimise** the empirical risk
- We **care about** the expected risk
- A flexible enough model drives the first to zero by memorising

That gap is **overfitting**

Which gives us the one rule for today

Hold out part of the data. Never let the fitting procedure touch it. Measure there.

When *not* to use machine learning

- The rule is **known** — write it
- The data is **not representative** of deployment
- Errors are **catastrophic and unexplainable**
- The data encodes an **injustice** you would automate
- A **simple baseline** already suffices

1943 — the first artificial neuron

McCulloch and Pitts: a neuron as a threshold unit.

Fires when the weighted sum of inputs passes a threshold.

1950 — Turing's imitation game

"Can machines think?" — replaced by a question you can test by observation.

1956 — Dartmouth

The term *artificial intelligence* is coined.

The proposal: ten people, two months, significant progress.

1957 — the perceptron learns

Rosenblatt: weights adjusted **from examples**, not set by hand.

Convergence theorem: if the data is linearly separable, it will find a separator.

1969 — the first winter

Minsky and Papert: a single-layer perceptron cannot represent XOR.

Correct. And narrower than the reception suggested.

1986 — backpropagation

An efficient way to train multi-layer networks.

The 1969 objection is answered. A second winter follows anyway.

1995–2010 — statistics takes over

- Support vector machines, ensembles, probabilistic methods
- From “simulating intelligence” to “estimating functions from data”

A retreat in ambition. An advance in rigour.

2012 — AlexNet

ImageNet error: 26% $\rightarrow$ 15%.

The architecture was recognisably LeNet, from 1989.

The pattern

Advance	Overclaim	Correction
Perceptron learns	Machines that walk and talk	XOR, first winter
Backprop trains depth	Networks solve everything	No data, second winter
Scale delivers	?	?

Three kinds of learning

The taxonomy divides by **where the target comes from** — not by algorithm.

Supervised

Each example carries a label a person produced.

- Category $\rightarrow$ **classification**
- Continuous $\rightarrow$ **regression**
- You can check answers against truth

Unsupervised

No targets at all. Find structure.

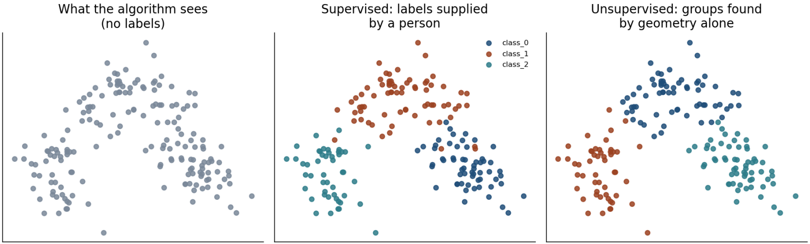
- Groups $\rightarrow$ clustering
- Fewer dimensions $\rightarrow$ dimensionality reduction
- Odd points $\rightarrow$ anomaly detection

Self-supervised

Hide part of the input. Predict it from the rest.

Nobody annotates anything — and the supervision is real.

One dataset, three questions



Where the target comes from

	Target	Cost	Measurable?
Supervised	A person	Expensive	Yes
Unsupervised	None	Free	Not directly
Self-supervised	The input	Free	Only the pretext

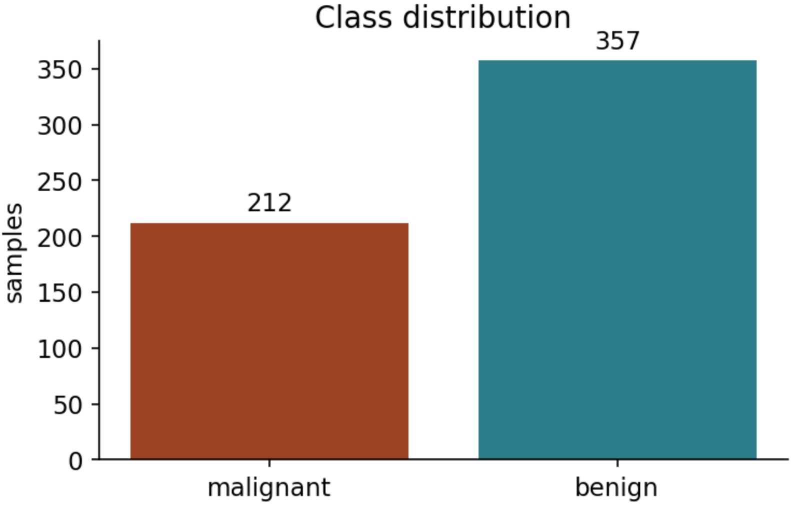
The workflow

1. Frame the problem
2. Inspect the data
3. **Split**
4. Baseline
5. Preprocess and model, in a pipeline
6. Evaluate with an honest metric
7. Diagnose
8. Iterate — with discipline

Step 1 — frame it

- What is predicted, from what?
- Who would use it, for what decision?
- **Which error is worse?**

Step 2 — look first



Step 3 — split, before anything else

Before scaling. Before selecting features. Before looking at correlations with the target.

Step 4 — baseline first

Predict the majority class. Nothing else.

63%

Everything from here is measured against that.

Step 5 — pipeline, not two steps

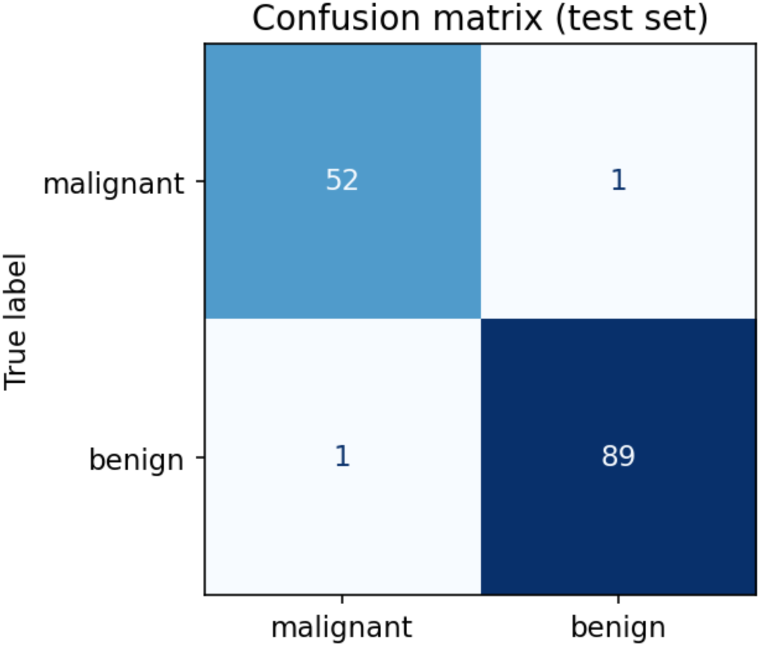
```
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000),
)
model.fit(X_train, y_train)
```

Step 6 — the result

- Baseline: 0.629
- Model: **0.986**

Done?

What accuracy hides



Precision and recall

- **Recall** (malignant): how many malignant tumours we caught
- **Precision** (malignant): how often an alert is correct
- They trade off against each other

What we did not do

- No tuning
- No comparison of model families
- No check of stability across splits

And: we never looked at the test set to make a decision.

Four ways a model misleads you

- Leakage
- Imbalance
- Shortcut features
- Single-split noise

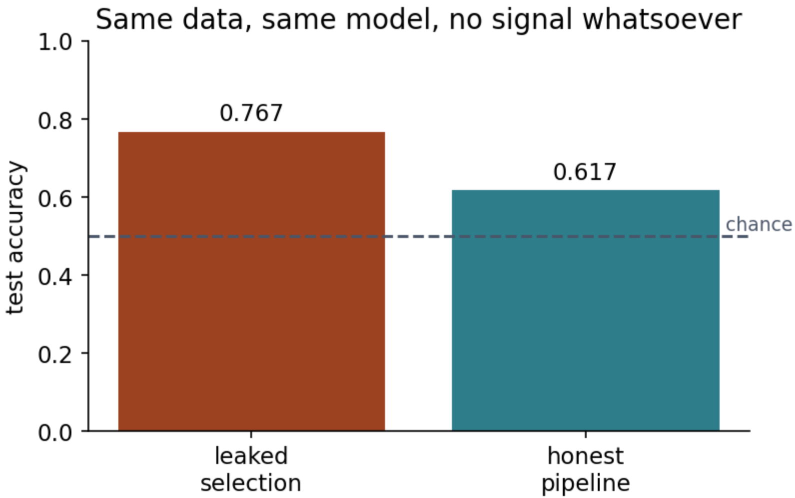
All produce numbers that pass a casual review.

200 samples. 5000 random features. Coin-flip labels.

There is nothing to learn.

Anything above 50% is an artefact.

Select features first, split second



The rule

Every step that **learns anything** must be fitted inside the training fold.

Scaling. Imputation. Encoding. Selection. Tuning.

99% accuracy, detecting nothing

1% positives. Answer "negative" every time.

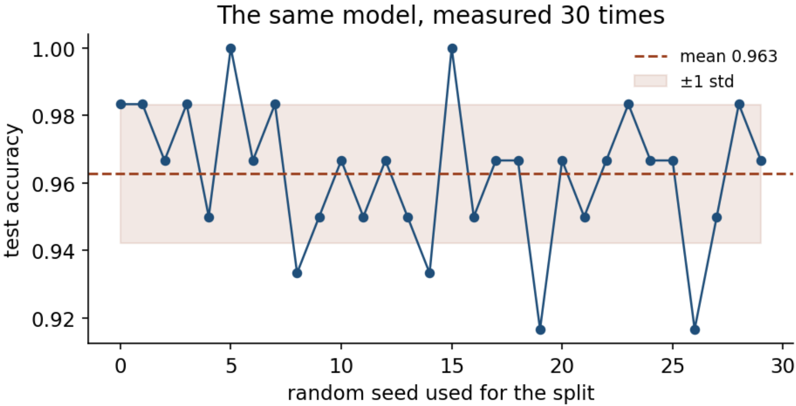
- Accuracy: **0.986**
- Recall on positives: **0.000**

Shortcut features

A column that is a **consequence** of the label, not a predictor of it.

`biopsy_scheduled` → accuracy rises → model is worthless

One split is one measurement



What the four have in common

None is a coding error.

All four run exactly as written and return a plausible number.

A model is not objective

- It is a compressed summary of its training data, **including its injustices**
- These methods find **correlation**, not cause
- Accuracy is not the only property that matters

Homework — due Friday 2 October

`Exercises/01_first_workflow.md`

Wine quality: run the workflow yourself, and **justify every decision**.

- No marks for accuracy
- Marks for methodology, and for saying what your number does not mean

Before next week

- Get the environment running
- Work the three notebooks in order
- Read the handout
- Take the quiz
- **Do the exercise**

Next: data — exploration, preparation, and leakage in earnest.